

LLD – User Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the User module of OpenScholar. The module manages user profile data, academic information, public researcher profiles, and social interactions such as following other users.

Authentication (login/register) is handled in the Auth module. This module focuses on user identity, relationships, and profile-level features.

2. Scope

The User module is responsible for:

- Retrieve authenticated user profile
- Update user profile information
- Store academic details (university, department)
- Retrieve user's papers (draft, pending, approved)
- Public user profile (view other researchers)
- Follow / unfollow users
- Retrieve followers and following lists

Out of Scope:

- Authentication (Auth module)
 - Paper submission logic (Paper module)
-

3. Data Model (Relevant Tables)

Tables used:

- users
- universities
- departments
- papers
- followers

users (extended fields)

- id (UUID)
- name (TEXT)

- email (TEXT)
- role (TEXT)
- status (ENUM: active | suspended)
- university_id (INT)
- department_id (INT)
- bio (TEXT, optional)
- profile_image_url (TEXT, optional)
- created_at (TIMESTAMP)

followers

- follower_id (UUID)
- following_id (UUID)
- created_at (TIMESTAMP)

Constraints:

- PRIMARY KEY (follower_id, following_id)
- FOREIGN KEY follower_id → users.id
- FOREIGN KEY following_id → users.id

4. API Contracts

4.1 Get Current User Profile

Endpoint: GET /api/users/me

Response: { "id": "uuid", "name": "John Doe", "email": "john@example.com", "role": "student", "status": "active", "bio": "...", "profileImageUrl": "...", "university": { "id": 1, "name": "XYZ" }, "department": { "id": 2, "name": "CS" } }

4.2 Update User Profile

Endpoint: PUT /api/users/me

Request Body: { "name": "Updated Name", "bio": "Researcher in AI", "profileImageUrl": "url", "universityId": 1, "departmentId": 2 }

4.3 Get User Papers

Endpoint: GET /api/users/me/papers

Query:

- status: draft | pending | approved
-

4.4 Public User Profile

Endpoint: GET /api/users/{id}

Response: { "id": "uuid", "name": "John", "bio": "...", "followersCount": 100, "followingCount": 50, "papers": [] }

4.5 Follow User

Endpoint: POST /api/users/{id}/follow

4.6 Unfollow User

Endpoint: DELETE /api/users/{id}/follow

4.7 Get Followers

Endpoint: GET /api/users/{id}/followers

4.8 Get Following

Endpoint: GET /api/users/{id}/following

5. Service Logic

5.1 getCurrentUser()

- Fetch user + relations

5.2 updateUserProfile()

- Validate
- Update fields

5.3 getUserPapers()

- Filter by status

- Join latest versions

5.4 followUser()

- Insert into followers (ignore duplicates)

5.5 unfollowUser()

- Delete relation
-

6. Validation Rules

- user must be authenticated
 - cannot follow self
 - prevent duplicate follow
 - university + department consistency
-

7. Security Considerations

- Users can only update their own profile
 - Admin can override (future)
 - Enforce JWT authentication
-

8. Performance Considerations

- Index on follower_id, following_id
 - Aggregate follower counts (future optimization)
-

9. Edge Cases

- User not found
 - Following self
 - Duplicate follow requests
 - Suspended users access restriction
-

10. Prisma Model Mapping

```
model User { id String @id @default(uuid()) name String email String @unique role String status String @default("active") bio String? profileImageUrl String? universityId Int? departmentId Int? }
```

```
model Follower { followerId String followingId String createdAt DateTime @default(now())
```

```
@@id([followerId, followingId]) }
```

11. Folder Structure

```
src/ app/api/users/me/route.ts app/api/users/[id]/route.ts app/api/users/[id]/follow/route.ts
```

```
modules/user/service.ts modules/user/repository.ts
```

12. Acceptance Criteria

- User can view and update profile
 - User can follow/unfollow others
 - Public profiles are accessible
 - Followers list is accurate
-

13. Future Enhancements

- Suggested users
 - Activity feed
 - Verified researcher badges
-

End of Document